

Course: Foundations of AI with Embeddings

Lecture 3: Recommender Systems

Collaborative filtering · SVD · Funk SVD · NMF

Liubov B.T., Vladimir Kukushkin

Constructor University, 2026

Motivating problem: Recommender Systems

A streaming platform has N users and p movies. We want to predict how many stars user i would give movie j .

Straightforward ML approach:

- Get *user features*: age, country, watch history, ...
- Get *movie features*: genre, director, cast, year, ...
- Train a regression model: $\hat{x}_{ij} = f(\text{user}_i, \text{movie}_j)$

Problem: feature engineering is expensive; many relevant factors are latent and hard to capture explicitly.

1. Collaborative filtering

User–item matrix

Collect all known ratings into a matrix $X \in \mathbb{R}^{N \times p}$:

	<i>Inception</i>	<i>Titanic</i>	<i>Alien</i>	<i>La La Land</i>	<i>Interstellar</i>
Alice	5	3	4	2	?
Bob	4	1	5	1	5
Carol	?	5	1	?	2
Dave	?	4	2	5	3

Most entries are *missing* — a typical user has rated $\ll 1\%$ of all movies.

Goal: fill in the ? entries.

Collaborative filtering

Core idea: users who agreed in the past will agree in the future.

User-based CF. To predict x_{ij} :

1. Find users most *similar* to user i (by their rating vectors)
2. Predict $\hat{x}_{ij}$ as a weighted average of their ratings of movie j

Item-based CF. Symmetrically – find movies similar to movie j , average over user i 's ratings of those movies.

Similarity (e.g. cosine over shared ratings):

$$\text{sim}(i, i') = \frac{\sum_{j \in \Omega_i \cap \Omega_{i'}} x_{ij} x_{i'j}}{\sqrt{\sum_j x_{ij}^2} \sqrt{\sum_j x_{i'j}^2}}.$$

Conditions: K nearest neighbours, $|\Omega_i \cap \Omega_{i'}| \geq M$.

Collaborative filtering: example

Task: predict Alice's rating for *Interstellar* using $K = 2$, $M = 3$.

Step 1: compute cosine similarity with Alice over commonly rated movies.

	<i>Inception</i>	<i>Titanic</i>	<i>Alien</i>	<i>La La Land</i>	<i>Interstellar</i>	$\text{sim}(\cdot, \text{Alice})$
Alice	5	3	4	2	?	–
Bob	4	1	5	1	5	0.93
Carol	–	5	1	–	2	–
Dave	–	4	2	5	3	0.83

Step 2: weighted average over $K = 2$ nearest neighbours (Bob, Dave):

$$\hat{x}_{\text{Alice, Interstellar}} = \frac{0.93 \times 5 + 0.83 \times 3}{0.93 + 0.83} = \frac{7.14}{1.76} \approx 4.1 \text{ stars.}$$

Problems

- *Cold start* – new users/items have no history.
- *Scalability* – pairwise similarity over millions of users is expensive.
- *Sparsity* – two users may share very few co-rated movies.
- *Cosine similarity*: $\{1, 2\}$ is the same as $\{2, 4\}$.

2. Singular Value Decomposition (SVD)

SVD: Definition

Any matrix $X \in \mathbb{R}^{N \times p}$ can be decomposed as

$$X = UDV^{\top},$$

where

- $U \in \mathbb{R}^{N \times N}$ – orthogonal ($U^{\top}U = I$), columns u_j are left singular vectors
- $D \in \mathbb{R}^{N \times p}$ – diagonal matrix of singular values $d_1 \geq d_2 \geq \dots \geq 0$
- $V \in \mathbb{R}^{p \times p}$ – orthogonal ($V^{\top}V = I$), columns v_j are right singular vectors

Full SVD and thin SVD

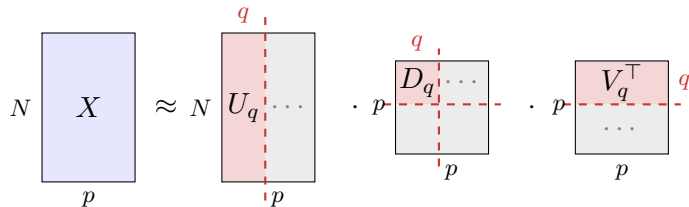
Full SVD: $N \times p$ matrix X is equal to the product of an $N \times N$ matrix U , an $N \times p$ diagonal matrix of singular values Σ (with zeros at the bottom), and a $p \times p$ matrix V^T .

$$X = U \Sigma V^T$$

Thin SVD: $N \times p$ matrix X is equal to the product of an $N \times p$ matrix U , a $p \times p$ diagonal matrix of singular values Σ , and a $p \times p$ matrix V^T .

$$X = U \Sigma V^T$$

Truncated SVD



The diagram illustrates the Truncated SVD decomposition of a matrix X . It shows the equation $X \approx U D V^T$ where U , D , and V are truncated to their first q components. Matrix X is a blue rectangle with dimensions N (rows) and p (columns). Matrix U is a tall rectangle with dimensions N (rows) and q (columns), with the first q columns highlighted in red. Matrix D is a square with dimensions q (rows) and q (columns), with the top-left $q \times q$ submatrix highlighted in red. Matrix V is a wide rectangle with dimensions q (rows) and p (columns), with the first q rows highlighted in red. Dashed red lines indicate the truncation at q for each matrix.

$$\begin{matrix} N \\ \end{matrix} \begin{matrix} X \\ \end{matrix} \begin{matrix} p \\ \end{matrix} \approx \begin{matrix} N \\ \end{matrix} \begin{matrix} U_q \cdots \\ \end{matrix} \begin{matrix} q \\ \end{matrix} \begin{matrix} D_q \cdots \\ \end{matrix} \begin{matrix} q \\ \end{matrix} \begin{matrix} V_q^T \cdots \\ \end{matrix} \begin{matrix} p \\ \end{matrix}$$

We keep only $q < p$ components.

Truncated SVD

Keep only the top q singular values – **best rank- q approximation** (Eckart–Young theorem):

$$X \approx X_q = U_q D_q V_q^\top,$$

where $U_q \in \mathbb{R}^{N \times q}$, $D_q \in \mathbb{R}^{q \times q}$, $V_q \in \mathbb{R}^{p \times q}$.

Eckart–Young theorem

X_q minimises $\|X - \hat{X}\|_F$ over all rank- q matrices $\hat{X}$.

- Columns of V_q are the same as principal components of PCA
- $U_q D_q$ gives the low-dimensional embedding
- Singular values d_k measure variance explained by each component

SVD and PCA: why V gives principal components

PCA objective. Given centred $X \in \mathbb{R}^{N \times p}$, find unit vector v maximising the variance of projections Xv :

$$\max_{\|v\|=1} \frac{\|Xv\|^2}{N} = \max_{\|v\|=1} v^\top \Sigma v, \quad \Sigma = \frac{X^\top X}{N}.$$

- Lagrangian $\mathcal{L} = v^\top \Sigma v - \lambda(v^\top v - 1) \implies \frac{\partial \mathcal{L}}{\partial v} = 2\Sigma v - 2\lambda v = 0$ gives $\Sigma v = \lambda v$.
- v must be an *eigenvector* of Σ .
- The variance equals $v^\top \Sigma v = \lambda$, which is maximised by the eigenvector with the *largest eigenvalue*.

Connection to SVD. For thin SVD $X = UDV^\top$ (centred X):

$$X^\top X V = V D U^\top U D V^\top V = V D^2$$

so columns of V are eigenvectors of $N\Sigma$ (principal directions)

Matrix factorisation: interpretation

Decompose $X \approx PQ^\top$, $q = 2$ latent factors — interpreted as **genres**.

$P_{N \times 2}$ — **user taste profile**

	<i>sci-fi</i>	<i>romance</i>
Alice	1.2	0.1
Bob	1.5	0.0
Carol	0.1	1.3
Dave	0.2	1.4

$Q_{p \times 2}$ — **movie genre profile**

	<i>sci-fi</i>	<i>romance</i>
Inception	1.3	0.0
Titanic	0.0	1.5
Alien	1.4	0.1
La La Land	0.0	1.2
Interstellar	1.1	0.2

Prediction: $\hat{x}_{ij} = p_i^\top q_j$ — dot product of user taste and movie profile.

E.g. Alice $\times$ Interstellar: $1.2 \cdot 1.1 + 0.1 \cdot 0.2 = 1.34$.

Key point: the latent factors are *not* given — they are *learned from data*.
The labels “sci-fi”, “romance” are our human interpretation after the fact.

3. Funk SVD

SVD and sparse matrices

- Problem: the user–item matrix X is *sparse*; a typical user rates $\ll 1\%$ of all items.
- Naive fix: impute missing values (global mean, or user/item mean), then apply SVD.
- Practical approach:
 - p_i and q_j are the user and item "profiles",
 - Prediction: $\hat{x}_{ij} = p_i^\top q_j$,
 - Optimise only on observed entries Ω .

Find low-rank factors $P \in \mathbb{R}^{N \times q}$, $Q \in \mathbb{R}^{p \times q}$ minimising

$$\min_{P, Q} \sum_{(i,j) \in \Omega} (x_{ij} - p_i^\top q_j)^2 + \lambda (\|P\|_F^2 + \|Q\|_F^2).$$

SGD for matrix factorisation

For each observed entry $(i, j) \in \Omega$ define the residual and per-entry loss:

$$e_{ij} = x_{ij} - p_i^\top q_j, \quad \ell_{ij} = e_{ij}^2 + \lambda(\|p_i\|^2 + \|q_j\|^2).$$

Gradients:

$$\frac{\partial \ell_{ij}}{\partial p_i} = -2e_{ij} q_j + 2\lambda p_i, \quad \frac{\partial \ell_{ij}}{\partial q_j} = -2e_{ij} p_i + 2\lambda q_j.$$

SGD update (step size η , factor 2 absorbed):

$$\begin{aligned} p_i &\leftarrow p_i + \eta (e_{ij} q_j - \lambda p_i), \\ q_j &\leftarrow q_j + \eta (e_{ij} p_i - \lambda q_j). \end{aligned}$$

Algorithm: iterate over observed ratings in random order, compute e_{ij} , update p_i and q_j .

Note

This is the *Simon Funk SVD* method, which influenced the Netflix Prize solutions (2006). Despite the name, no SVD is computed explicitly.

SVD++

Motivation. Funk SVD ignores two useful signals:

- *Biases* – some users rate high on average; some movies are universally popular.
- *Implicit feedback* – the *set* of items a user rated reveals preferences itself.

SVD++ (Koren 2008) adds implicit item vectors y_k – one per item – to augment the user factor:

$$\hat{x}_{ij} = \mu + b_i + b_j + q_j^\top \left(p_i + |\Omega_i|^{-1/2} \sum_{k \in \Omega_i} y_k \right), \text{ where}$$

- μ is the global mean rating,
- b_i is the user bias,
- b_j is the item bias,
- y_k is the implicit user feedback,
- Ω_i is the set of items rated by user i .

Matrix factorisation vs. collaborative filtering

	CF (kNN)	Matrix factorisation
Scalability	$O(N^2)$ similarity matrix	$O(\Omega \cdot q)$ per epoch
Sparsity	fails if $ \Omega_i \cap \Omega_{i'} $ small	latent factors generalise globally
Biases	not modelled	explicit b_i, b_j, μ terms
Implicit data	ignored	y_k vectors in SVD++
Accuracy	baseline	state-of-the-art (Netflix Prize)

Key insight. Instead of asking “*who is similar to this user?*”, matrix factorisation asks “*what latent factors explain all ratings at once?*”

4. NMF decomposition

Idea

Assume $X \geq 0$. We want to find a decomposition

$$X_{N \times p} \approx W_{N \times q} H_{q \times p},$$

where $w_{ik} \geq 0$, $h_{kj} \geq 0$.

- Assume that $x_{ij} \sim \text{Pois}(\lambda_{ij})$, $\lambda_{ij} = (WH)_{ij} = \sum_k w_{ik} h_{kj}$
- Likelihood $l(W, H) = \prod_{i,j} \frac{e^{-\lambda_{ij}} \lambda_{ij}^{x_{ij}}}{x_{ij}!}$
- $\log l(W, H) = \underbrace{\sum_{i,j} x_{ij} \log \lambda_{ij} - \lambda_{ij} - \log(x_{ij}!)}_{L(W,H)} \xrightarrow{w_{ij}, h_{ij}} \max$

$$L(W, H) = \sum_{i,j} x_{ij} \log \lambda_{ij} - \lambda_{ij} \xrightarrow{w_{ij}, h_{ij}} \max$$

- $\frac{\partial L}{\partial w_{ik}} = \sum_j \left(\frac{x_{ij}}{\lambda_{ij}} \frac{\partial \lambda_{ij}}{\partial w_{ik}} - \frac{\partial \lambda_{ij}}{\partial w_{ik}} \right) = \underbrace{\sum_j \frac{x_{ij}}{\lambda_{ij}} h_{kj}}_{A_{ik}} - \underbrace{\sum_j h_{kj}}_{B_{ik}},$
- $\frac{\partial L}{\partial h_{kj}} = \sum_i \left(\frac{x_{ij}}{\lambda_{ij}} \frac{\partial \lambda_{ij}}{\partial h_{kj}} - \frac{\partial \lambda_{ij}}{\partial h_{kj}} \right) = \sum_i \frac{x_{ij}}{\lambda_{ij}} w_{ik} - \sum_i w_{ik}$
- Due to $w_{ik} \geq 0, h_{kj} \geq 0$ we cannot simply use gradient descent.
- Lee and Seung 2001:

$$w_{ik} \leftarrow w_{ik} \frac{A_{ik}}{B_{ik}} = \frac{\sum_j h_{kj} x_{ij} / (WH)_{ij}}{\sum_j h_{kj}}$$

$$h_{kj} \leftarrow h_{kj} \frac{C_{kj}}{D_{kj}} = \frac{\sum_i w_{ik} x_{ij} / (WH)_{ij}}{\sum_i w_{ik}}$$

References

-  T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*. Springer, 2nd ed., 2009.
-  C. Eckart, G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
-  S. Funk. Netflix Update: Try This at Home. *Blog post*, 2006.
<https://sifter.org/simon/journal/20061211.html>
-  Y. Koren. Factorization Meets the Neighborhood: a Multifaceted Collaborative Filtering Model. *Proceedings of the 14th ACM SIGKDD*, pp. 426–434, 2008.
-  D. D. Lee, H. S. Seung. Algorithms for Non-negative Matrix Factorization. *Advances in Neural Information Processing Systems (NIPS)*, 2001.